Code guide — how the project is organised and how to run a simulation

Everything lives in one MATLAB folder. The Simulink model `Final.slx` is the plant + controller; all MATLAB code around it builds its inputs, runs it and plots the results.

Golden rule: the model reads **workspace variables**, and those variables are produced by `harness.config` (parameters) and `harness.input` (scenario). You never set model parameters by hand — you build a config struct and pass it to a scenario.

---

1. Where things live

Path	What is in it
`Final.slx`	The model: plant (actuators + UAV), controller (position, attitude planner, attitude, allocation), adaptive block
`+uav/`	Plant definition: `params` (all physical constants), `with_package`, `trim`, `allocation`, `propagate`
`+design/`	Controller gains: `position`, `attitude`, `adaptive`
`+harness/`	Running machinery: `config`, `input`, `scenario`, `unpack`, `metrics`, `report`, `fastrestart`, `model_vars`, and the Monte Carlo helpers
`+traj/`	References: `setpoint`, `helix`, `helix_old`, `mission`, `slalom`, `multisine3d`, plus `pack` / `pos` / `vel` / `acc` / `yaw`
`+disturb/`	Disturbances: `gust`, `step`, `constant`, `fault`, `none`
`+viz/`	Plotting: one function per figure type, plus `defaults` (style) and `export` (save all figures)
`run_task*.m`	One script per exam task; these are the entry points
`run_montecarlo_final.m`	The Task 3.4 campaign (crash-safe, resumable)
`results/`	Saved Monte Carlo `.mat` files
`Figures/`	Exported figures, one subfolder per task

---

2. Which file holds which parameter

Physical vehicle — `+uav/params.m` mass 1.2 kg, 6 rotors, inertia, arm length, thrust coefficient `k_f = 3.65e-6`, motor time constant (`k_m = 1/0.05`), rotor limits `Omega_min/Omega_max` (132–1079 rad/s), drag `D_fv = diag(0.26, 0.26, 0)` and `D_tauomega`, allocation matrices `F1`, `F2`.

Package — `+uav/with_package.m`, called by `harness.config` Combines the vehicle and a cube payload: total mass, combined centre of mass, inertia with the parallel-axis term.

Controller gains — `+design/`

- `position.m`: the outer-loop PID gains, derived from bounds (`max_pos_err`, `max_vel`, `max_int_err`, `max_force`).
- `attitude.m`: the inner-loop gains (`K_R`, `K_omega`); `harness.config(attitude_gains=...)` picks `'designed'` or `'as_flown'`.
- `adaptive.m`: everything PBMRAC — `L_p`, `L_v`, `Gamma_theta`, `Gamma_lambda`, `sigma_e`, and the projection set (`lambda_proj_center/r0/r1`).

Scenario parameters — `+harness/config.m` Package mass and offset, adaptive nominal mass, the `startup` option (below), and all default flags and initial conditions (`MRAC_on`, `package_exist`, `robust_on`, `p_i_0`, `v_i_0`, `q_0`, `omegab_0`, `vb_0`, `Omega_prop_0`, `T_prop_0`).

What the model actually reads — `+harness/model_vars.m` The authoritative list of workspace variables the model needs. If you add a variable to the model, add it here too, or the run will fail with an undefined-variable error.

---

3. Initial conditions

Every state starts at zero unless you change it. The defaults in `harness.config`:

Variable	Default	Meaning
`p_i_0`	[0 0 0]	inertial position; also seeds the predictor
`v_i_0`	[0 0 0]	inertial velocity
`q_0`	[1 0 0 0]	attitude quaternion (level)
`omegab_0`	[0 0 0]	body angular rate
`vb_0`	[0 0 0]	body linear velocity
`Omega_prop_0`	[0 0 0 0 0 0]	**rotor speeds** (actuator Integrator IC)
`T_prop_0`	[0 0 0 0 0 0]	thrust Transport Delay initial output, per rotor

The `startup` option (from Antonio's version). Two model fields must reference these variables:

Block	Parameter	Value
`actuators / actuators / Integrator`	Initial condition	`Omega_prop_0`
`actuators / thrust / Transport Delay`	Initial output	`T_prop_0`

```
P = harness.config();                          % startup = 'stopped' -> Omega_prop_0 = 0
P = harness.config(startup = 'nominal_hover');  % Omega_prop_0 = 733.2 rad/s on all six rotors
```

- `'stopped'` (default): rotors start stopped. They spin up with the 0.05 s motor time constant, so for roughly the first 0.2 s the vehicle produces almost no lift and sags. That is the large transient at t = 0 in most plots.
- `'nominal_hover'`: rotors start at the **Task 1.2 hover trim**, `sqrt(m·g / (N_r·k_f)) = 733.2 rad/s`, the speed at which the six rotors carry the bare 1.2 kg vehicle (11.772 N). It always uses the **bare** mass: with

a package the true trim is higher and a small sag remains by design (the package is unknown to the controller).

`T_prop_0` is set by `harness.input`, not by hand: `T_prop_0 = λ(0) · k_f · Omega_prop_0²`, i.e. the thrust the rotors are already producing, using the first sample of the rotor-effectiveness signal. The 10 ms actuator delay therefore outputs that thrust from t = 0 instead of zero, so a hover start has no thrust dropout. For a custom start, set `P.Omega_prop_0` (a 6-by-1 vector) before calling `harness.scenario`.

Position, velocity and heading are always taken from the first sample of the reference by `harness.input`; the start-up mode only sets the rotors.

---

4. Running one simulation

The pattern is always the same three lines:

```
ref = traj.helix(150);                              % 1. a reference
P   = harness.config(package_mass = 0.4);           % 2. parameters
R   = harness.scenario(ref, config = P, ...         % 3. run it
                       package = 1, mrac = 1, robust = 1, label = 'PBMRAC');
```

`harness.scenario` calls `harness.input` (build the `Simulink.SimulationInput`), runs `sim`, and returns an unpacked result struct `R` with fields `t`, `p`, `p_d`, `v`, `v_d`, `euler`, `omegab`, `T_c`, `T_out`, `w_a`, `lambda_hat`, and so on — plus a `_t` time vector for each logged signal.

Scenario options (`harness.scenario` / `harness.input`):

Option	Meaning
`config`	the struct from `harness.config`
`package`	0/1 — attach the payload in the **plant** (the controller never knows)
`mrac`	0/1 — enable the adaptive augmentation
`robust`	0/1 — use the robust update laws (e-modification + projection)
`wind`	wind **velocity** signal, e.g. `disturb.gust(...)`
`force`	wind **force** signal
`disturbance`	external force in the inertial frame, e.g. `disturb.step(...)`
`lambda`	rotor-effectiveness signal, e.g. `disturb.fault(...)`
`stop_time`	override the simulation end time (default: the reference's)
`label`	a name used in reports and plots

References (`+traj/`):

```
ref = traj.setpoint([0;0;3], [3;0;3], 35);   % step from p0 to pf over T seconds
ref = traj.helix(150, radius=10, rate=0.2, climb=2, approach_time=20);
ref = traj.helix_old(150);                     % the pre-approach helix (Task 2 figures)
[ref, M] = traj.mission();                     % the Task 4 delivery mission (+ planning info)
```

Disturbances (`+disturb/`), all built on the reference's own time vector:

```
wind = disturb.gust(ref.time, t0=60, duration=20, peak=[8;0;0]);   % 1-cosine gust [m/s]
lam  = disturb.fault(ref.time, t0=90, lambda=[0.8;0.6;0.75;0.78;0.5;0.9]);
dist = disturb.step(ref.time, t0=60, value=[10;0;0]);              % inertial force [N]
```

Metrics and plots:

```
m = harness.metrics(R);                       % rms_p, peak_p, ss_p, rms_v, ...
harness.report({R_bl, R_ad}, {'Baseline','PBMRAC'}, title='My comparison');
viz.defaults();                               % project plot style (do this first)
viz.errors({R_bl, R_ad}, {'Baseline','PBMRAC'}, name='Error norms');
viz.tracking(...); viz.actuators(...); viz.attitude(...); viz.path3d(...);
viz.export('MyTask');                         % saves every open figure to Figures/MyTask
```

Many runs in a row: hold fast restart open so the model is not recompiled each time:

```
guard = harness.fastrestart('Final');   % compile once
% ... many harness.scenario calls ...
clear guard                             % release before plotting
```

---

5. The task scripts

Script	What it does
`run_task1_trim.m`	Hover and cruise trim: analytic, numerical (`uav.trim`) and open-loop check
`run_task2_setpoints.m`	Baseline controller: lateral/vertical steps, helix, force disturbance, gust, rotor fault
`run_task3_package_final.m`	Adaptive controller: package effect, PBMRAC, payload sweep, fault, fault + gust
`run_task4_mission.m`	Delivery mission: planning + baseline vs PBMRAC under gust and fault
`run_montecarlo_final.m`	The 500-vehicle campaign
`control_Design.m`	Loop shaping: design plants, Bode plots, margins, step responses
`plot_montecarlo_final_polished_v3.m`	Re-plots a saved campaign, no simulation

Each script ends with `viz.export('TaskName')`, which **empties** `Figures/TaskName` and writes every open figure there, numbered in creation order.

One model for every task. Task 4 flies `Final.slx` too. `harness.build_single_model` (run once; it backs the model up to `backup/Final_before_single_model.slx`) put the Task 4 blocks into it, and two parameters switch them on — `harness.config` defaults leave them off, and Tasks 1–3 were checked to be bit-identical with and without the blocks:

Parameter	Default (Tasks 1–3)	Task 4	Effect
`task4_ground_z`	`-Inf` (no floor)	pad height	floor limit, impact reset and ground support become active
`pb_flight_gate`	`0`	`1`	variant control of the **PBMRAC enable** Variant Subsystem: 0 = `MRAC_on` straight to the adaptive Enable; 1 = measured flight gate AND no contact. The inactive choice is removed at compile time, so the gate's discrete 0.01 s rate never reaches Tasks 1–3
`pb_gate_Ts`, `pb_gate_cfg`	0.01 s, [0; 0.15; 0.02; 0.1]	set by the script	gate sample time and [ground_z; height; climb rate; confirm time]

The force logs `f_bl`, `f_ad`, `f_total` are at the root (they now cover the whole run, also for the baseline), next to `pb_active`, `pb_flight_phase`, `ground_contact` and the rotor logs `Omega_cmd`, `Omega_limited`, `Omega_actual`. The Task 4 pieces (from Antonio's Task 4):

- **Ground contact** (`build_single_model`, `add_task4_ground_block`, `add_task4_impact`, `task4_ground_support`): the floor limits the height, stops the vehicle on impact (ideal sticking landing) and carries it at rest until the rotors lift it off. No bounce, sliding or landing-gear model.
- **PBMRAC flight gate** (`task4_flight_gate`): PBMRAC switches on only after a measured lift-off (height ≥ 0.15 m and climb ≥ 0.02 m/s for 0.1 s) and off at touchdown; the baseline flies throughout.
- **Takeoff hover** (`S.hover_h`, `S.rise_t`, `S.hover_t` in `run_task4_mission`): the reference rises from the pad to 1 m in 2 s and hovers 5 s before the mission (planned with `traj.mission(..., takeoff_alt = 1)`); PBMRAC switches on during the rise and converges in the hover.
- **End hold**: the reference stays on the pad for 10 s after the planned touchdown, sitting 5 cm below it in the first second so that every controller actually makes contact.
- **Reports**: `task4_flight_report` (lift-off, PBMRAC on/off, contact, per-phase RMS including the hold) and `task4_motor_report` (rotor request, clipped request, actual speed).

`test_task4_flight_gate` and `test_task4_contact` run first, in seconds, before the mission is planned.

---

6. Monte Carlo

```
MC = run_montecarlo_final(n = 50);                                    % quick check
MC = run_montecarlo_final;                                            % full campaign
MC = run_montecarlo_final(resume = 'results/montecarlo_final_....mat'); % continue
```

It checkpoints into `results/` as it goes, so a crash costs only the current run. `resume` only accepts files produced by this runner (it rebuilds everything from the settings stored inside). To re-plot a finished campaign without simulating, set `MAT_FILE` at the top of `plot_montecarlo_final_polished_v3.m`.

---

7. Adding something

- **A new reference:** write `+traj/mine.m` returning `traj.pack(t, p, v, a, yaw)`.

- **A new disturbance:** write `+disturb/mine.m` returning `struct('time', t, 'signals', struct('values', V, 'dimensions', n))`.
- **A new model variable:** add it to `harness.config` (a default) **and** to `harness.model_vars` (the list pushed into the model).
- **A new logged signal:** log it in the model, then add its name to the `OPTIONAL` list in `harness.unpack`.

---

8. Things that will bite you

- **Signal names must match the model.** `harness.unpack` expects `e_hat_pred` / `theta_a_hat_pred` (lower case). If the model logs different names, the fields simply go missing from `R`.
- **`viz.export` cleans its folder first.** If a run crashes between the clean and the export, the folder is left empty — re-run the script.
- **Many open figures crash MATLAB** on long scripts. Run with `set(groot,'defaultFigureVisible','off')` and close figures as you export them.
- **`traj.helix` changed.** It now starts from rest and approaches the helix; the old constant-radius version is `traj.helix_old`. Figures made before the change cannot be reproduced with the new default.
- **The model name is `Final`.** Scenario and fast restart default to it.
- **`clear guard` before plotting**, or the model stays in fast-restart mode and figures may lag.